

# Assignment 10: Data Scraping

Isabelle Kuehn

## OVERVIEW

This exercise accompanies the lessons in Environmental Data Analytics on data scraping.

## Directions

1. Rename this file `<FirstLast>_A10_DataScraping.Rmd` (replacing `<FirstLast>` with your first and last name).
2. Change “Student Name” on line 3 (above) with your name.
3. Work through the steps, **creating code and output** that fulfill each instruction.
4. Be sure your code is tidy; use line breaks to ensure your code fits in the knitted output.
5. Be sure to **answer the questions** in this assignment document.
6. When you have completed the assignment, **Knit** the text and code into a single PDF file.

## Set up

1. Set up your session:
  - Load the packages `tidyverse`, `rvest`, and any others you end up using.
  - Check your working directory

```
#1
library(tidyverse)
library(rvest)
library(lubridate)
library(viridis)
library(here)

getwd()
```

```
## [1] "/home/guest/R/RemoteRepositoryClone"
```

2. We will be scraping data from the NC DEQs Local Water Supply Planning website, specifically the Durham’s 2024 Municipal Local Water Supply Plan (LWSP):
  - Navigate to <https://www.ncwater.org/WUDC/app/LWSP/search.php>
  - Scroll down and select the LWSP link next to Durham Municipality.
  - Note the web address: <https://www.ncwater.org/WUDC/app/LWSP/report.php?pwsid=03-32-010&year=2024>

Indicate this website as the as the URL to be scraped. (In other words, read the contents into an `rvest` webpage object.)

```
#2
LWSP_URL <-
  read_html('https://www.ncwater.org/WUDC/app/LWSP/report.php?pwsid=03-32-010&year=2024')
```

3. The data we want to collect are listed below:

- From the “1. System Information” section:
- Water system name
- PWSID
- Ownership
- From the “3. Water Supply Sources” section:
- Maximum Day Use (MGD) - for each month

In the code chunk below scrape these values, assigning them to four separate variables.

HINT: The first value should be “Durham”, the second “03-32-010”, the third “Municipality”, and the last should be a vector of 12 numeric values (represented as strings)“.

```
#3
water_sys_name <- LWSP_URL %>%
  html_nodes("div+ table tr:nth-child(1) td:nth-child(2)") %>%
  html_text()
PWSID <- LWSP_URL %>%
  html_nodes("td tr:nth-child(1) td:nth-child(5)") %>%
  html_text()
ownership <- LWSP_URL %>%
  html_nodes("div+ table tr:nth-child(2) td:nth-child(4)") %>%
  html_text()
MDU <- LWSP_URL %>%
  html_nodes("th~ td+ td") %>%
  html_text()
```

4. Convert your scraped data into a dataframe. This dataframe should have a column for each of the 4 variables scraped and a row for the month corresponding to the withdrawal data. Also add a Date column that includes your month and year in data format. (Feel free to add a Year column too, if you wish.)

TIP: Use `rep()` to repeat a value when creating a dataframe.

NOTE: It’s likely you won’t be able to scrape the monthly withdrawal data in chronological order. You can overcome this by creating a month column manually assigning values in the order the data are scraped: “Jan”, “May”, “Sept”, “Feb”, etc...

5. Create a line plot of the maximum daily withdrawals across the months for 2024, making sure, the months are presented in proper sequence.

```

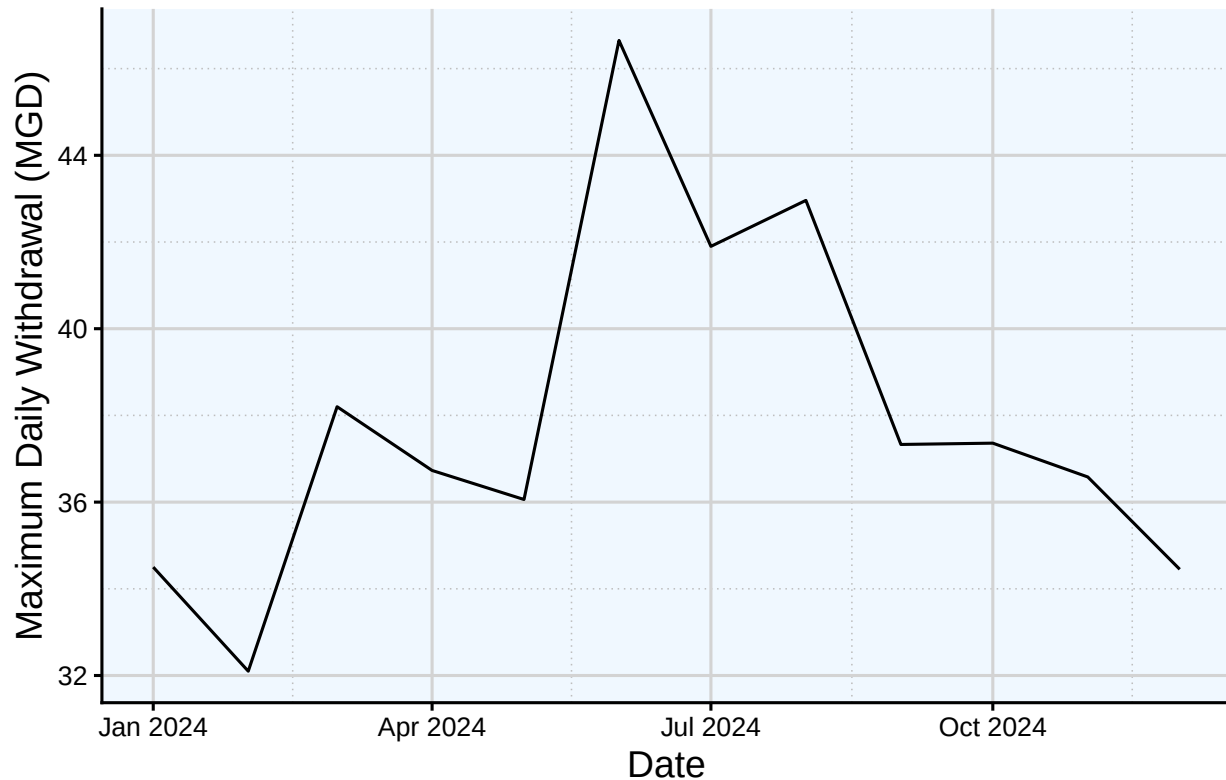
#4
LWSP_df <- data.frame(
  "Water System Name" = water_sys_name,
  "PWSID" = PWSID,
  "Ownership" = ownership,
  "Maximum Day Use" = as.numeric(MDU),
  "Month" = c("Jan", "May", "Sept", "Feb", "Jun", "Oct", "Mar", "Jul", "Nov", "Apr",
              "Aug", "Dec"),
  "Year" = rep(2024, 12)) %>%
  mutate(Date = my(paste(Month, "-", Year)))

#5
default_theme <- theme_classic(base_size = 12) +
  theme(axis.text = element_text(color = "black"),
        axis.title.x = element_text(color = "black", size = 14),
        axis.title.y = element_text(color = "black", size = 14),
        plot.title = element_text(color = "black", size = 16),
        plot.background = element_rect(fill = "white"),
        panel.background = element_rect(fill = "aliceblue"),
        panel.grid.major = element_line(color = "lightgrey"),
        panel.grid.minor = element_line(color = "grey", linetype = "dotted"),
        legend.text = element_text(color = 'black', size = 9),
        legend.title = element_text(size = 10),
        legend.position = "top")
theme_set(default_theme)

ggplot(LWSP_df, aes(x=Date, y=Maximum.Day.Use)) +
  geom_line() +
  labs(title = "Daily Water Use in 2024 for the Durham Municipality",
       y="Maximum Daily Withdrawal (MGD)",
       x="Date")

```

## Daily Water Use in 2024 for the Durham Municipality



6. Note that the PWSID and the year appear in the web address for the page we scraped. Construct a function with two inputs - “PWSID” and “year” - that:

- Creates a URL pointing to the LWSP for that PWSID for the given year
- Creates a website object and scrapes the data from that object (just as you did above)
- Constructs a dataframe from the scraped data, mostly as you did above, but includes the PWSID and year provided as function inputs in the dataframe.
- Returns the dataframe as the function’s output

```
#6.
pwsid <- "03-32-010"
the_year <- 2024
scrape.LWSP <- function(the_year, pwsid){

  the_base_url <- 'https://www.ncwater.org/WUDC/app/LWSP/report.php?'
  the_URL <- paste0(the_base_url, 'pwsid=', pwsid, '&year=',
                    the_year)
  lwsp_URL <- read_html(the_URL)
  print(lwsp_URL)

  water_sys_name_tag <- 'div+ table tr:nth-child(1) td:nth-child(2)'
  PWSID_tag <- 'td tr:nth-child(1) td:nth-child(5)'
  ownership_tag <- 'div+ table tr:nth-child(2) td:nth-child(4)'
  MDU_tag <- 'th~ td+ td, th~ td+ td'
```

```

system_name <- lwsp_URL %>% html_nodes(water_sys_name_tag) %>% html_text()
PWSID_name <- lwsp_URL %>% html_nodes(PWSID_tag) %>% html_text()
ownership_name <- lwsp_URL %>% html_nodes(ownership_tag) %>% html_text()
MDU_withdrawal <- lwsp_URL %>% html_nodes(MDU_tag) %>% html_text()

df_MDU <- data.frame("Month" = c("Jan", "May", "Sept", "Feb", "Jun", "Oct", "Mar",
                                "Jul", "Nov", "Apr", "Aug", "Dec"),
                    "Year" = rep(the_year, 12),
                    "Maximum.Day.Use" = as.numeric(MDU_withdrawal)) %>%
  mutate(Water.System.Name = !!system_name,
         PWSID = !!pwsid,
         Ownership = !!ownership_name,
         Date = my(paste(Month, "-", Year)))

return(df_MDU)
}

```

7. Use the function above to extract and plot max daily withdrawals for Durham (PWSID='03-32-010') for each month in 2020. Then show the structure of the dataframe with either the `str()` or the `glimpse()` function.

```

#7
durham_2020 <- scrape.LWSP(2020, "03-32-010")

## {html_document}
## <html xmlns="http://www.w3.org/1999/xhtml" lang="en" xml:lang="en">
## [1] <head>\n<title>DWR :: Local Water Supply Planning</title>\n<meta http-equi ...
## [2] <body id="plan">\r\n<!--<div id="division-header">\r\n<a name="top" href= ...

view(durham_2020)

glimpse(durham_2020)

```

```

## Rows: 12
## Columns: 7
## $ Month      <chr> "Jan", "May", "Sept", "Feb", "Jun", "Oct", "Mar", "J-
## $ Year        <dbl> 2020, 2020, 2020, 2020, 2020, 2020, 2020, 2020, 2020~
## $ Maximum.Day.Use <dbl> 36.01, 36.98, 41.69, 32.05, 40.61, 40.56, 37.29, 43.~
## $ Water.System.Name <chr> "Durham", "Durham", "Durham", "Durham", "Durham", "D~
## $ PWSID       <chr> "03-32-010", "03-32-010", "03-32-010", "03-32-010", ~
## $ Ownership   <chr> "Municipality", "Municipality", "Municipality", "Mun~
## $ Date        <date> 2020-01-01, 2020-05-01, 2020-09-01, 2020-02-01, 202~

```

8. Use the function above to extract data for Cary (PWSID = '03-92-020') in 2020. Combine this data with the Durham data collected above and create a plot that compares Cary's to Durham's water withdrawals.

```

#8
the_pwsids <- c("03-92-020", "03-32-010")
the_years <- rep(2020)

```

```

params <- expand.grid(the_year = the_years, pwsid = the_pwsids)

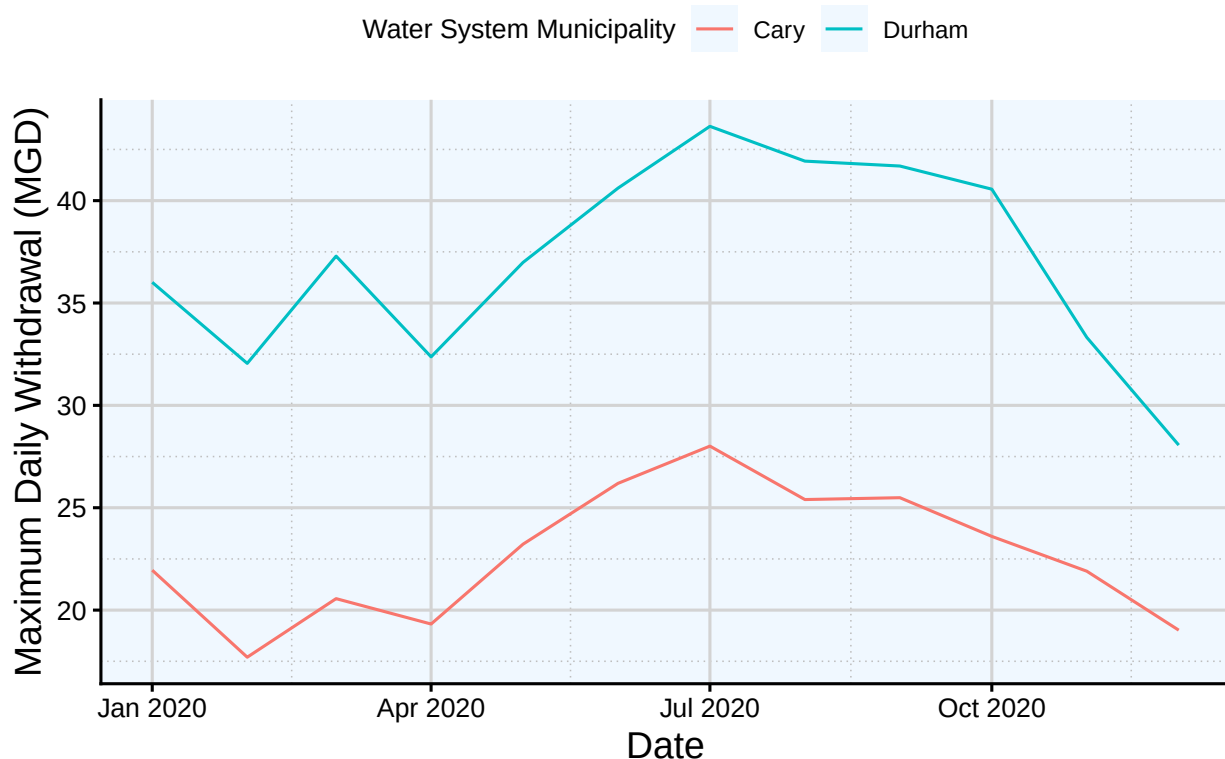
df_MDU <- pmap(params, scrape.LWSP) %>% bind_rows()

## {html_document}
## <html xmlns="http://www.w3.org/1999/xhtml" lang="en" xml:lang="en">
## [1] <head>\n<title>DWR :: Local Water Supply Planning</title>\n<meta http-equ ...
## [2] <body id="plan">\r\n<!--<div id="division-header">\r\n<a name="top" href= ...
## {html_document}
## <html xmlns="http://www.w3.org/1999/xhtml" lang="en" xml:lang="en">
## [1] <head>\n<title>DWR :: Local Water Supply Planning</title>\n<meta http-equ ...
## [2] <body id="plan">\r\n<!--<div id="division-header">\r\n<a name="top" href= ...

ggplot(df_MDU,aes(x=Date,y=Maximum.Day.Use, color = Water.System.Name)) +
  geom_line() +
  labs(title = "2020 Maximum Daily Withdrawal for Durham and Cary",
        y="Maximum Daily Withdrawal (MGD)",
        x="Date",
        color = "Water System Municipality")

```

## 2020 Maximum Daily Withdrawal for Durham and Cary



9. Use the code & function you created above to plot Cary’s max daily withdrawal by months for the years 2018 thru 2023. Add a smoothed line to the plot (method = ‘loess’).

TIP: See Section 3.2 in the “10\_Data\_Scraping.Rmd” where we apply “map2()” to iteratively

run a function over two inputs. Pipe the output of the `map2()` function to `bind_rows()` to combine the dataframes into a single one, and use that to construct your plot.

```
#9
total_years <- c(2018:2023)
the_PWSIDS <- rep("03-92-020")

dfs_MDU_cary <- map2(total_years, the_PWSIDS, scrape.LWSP)

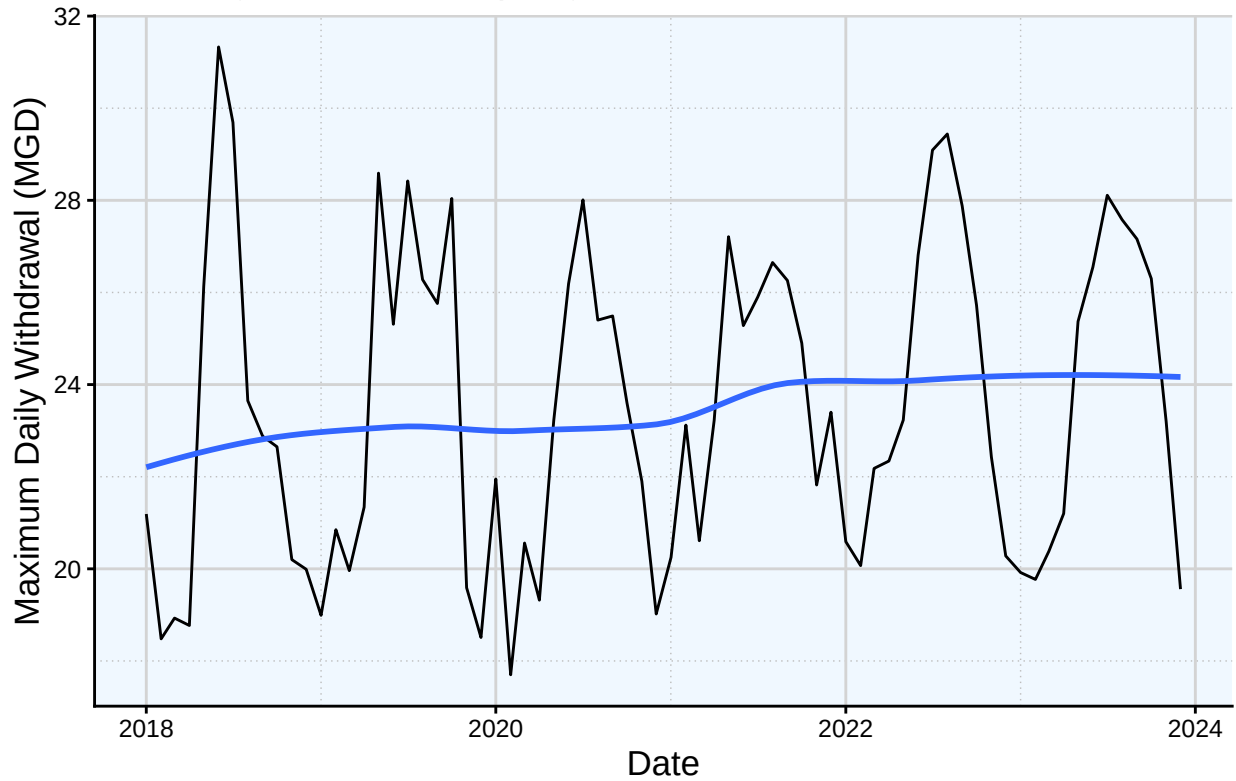
## {html_document}
## <html xmlns="http://www.w3.org/1999/xhtml" lang="en" xml:lang="en">
## [1] <head>\n<title>DWR :: Local Water Supply Planning</title>\n<meta http-equ ...
## [2] <body id="plan">\r\n<!--<div id="division-header">\r\n<a name="top" href= ...
## {html_document}
## <html xmlns="http://www.w3.org/1999/xhtml" lang="en" xml:lang="en">
## [1] <head>\n<title>DWR :: Local Water Supply Planning</title>\n<meta http-equ ...
## [2] <body id="plan">\r\n<!--<div id="division-header">\r\n<a name="top" href= ...
## {html_document}
## <html xmlns="http://www.w3.org/1999/xhtml" lang="en" xml:lang="en">
## [1] <head>\n<title>DWR :: Local Water Supply Planning</title>\n<meta http-equ ...
## [2] <body id="plan">\r\n<!--<div id="division-header">\r\n<a name="top" href= ...
## {html_document}
## <html xmlns="http://www.w3.org/1999/xhtml" lang="en" xml:lang="en">
## [1] <head>\n<title>DWR :: Local Water Supply Planning</title>\n<meta http-equ ...
## [2] <body id="plan">\r\n<!--<div id="division-header">\r\n<a name="top" href= ...
## {html_document}
## <html xmlns="http://www.w3.org/1999/xhtml" lang="en" xml:lang="en">
## [1] <head>\n<title>DWR :: Local Water Supply Planning</title>\n<meta http-equ ...
## [2] <body id="plan">\r\n<!--<div id="division-header">\r\n<a name="top" href= ...

cary_18_to_23_df <- bind_rows(dfs_MDU_cary)

ggplot(cary_18_to_23_df, aes(x=Date, y=Maximum.Day.Use)) +
  geom_line() +
  geom_smooth(method="loess", se=FALSE) +
  labs(title = "2018-2023 Maximum Daily Withdrawal for the
    Cary Water Municipality",
    y="Maximum Daily Withdrawal (MGD)",
    x="Date")

## 'geom_smooth()' using formula = 'y ~ x'
```

### 2018–2023 Maximum Daily Withdrawal for the Cary Water Municipality



Question: Just by looking at the plot (i.e. not running statistics), does Cary have a trend in water usage over time?

Answer: Looking singularly at the plot, from a yearly scale it appears that Cary has an annual trend in water usage based on the patterned peaks across the five years. This would make sense as water use is typically higher during hotter months, and would cycle in this pattern every year. From the smoothed line plotted against the graph, it also appears that a small positive trend exists. Maximum daily withdrawal of water in Cary is increasing across years, rising from an average of around 22 MGD in early 2018 to above 24 MGD in late 2023.